

MATUNDU PRIMARY SCHOOL MANAGEMENT SYSTEM

IMPLEMENTATION PLAN

Student Name	Musa Muthami Mwange
Registration No.	23/05037
Course	STU 4201 — Final Year Project II
Supervisor	Deborah Mbula
Institution	KCA University
Trimester	Trimester 3, 2026
Stream	A (PTDL)
Project Title	Matundu Primary School Management System (MATUNDU-SMS)
Document Version	1.0
Date	October 2026

TABLE OF CONTENTS

1. [Introduction](#)
2. [System Overview](#)
3. [Implementation Strategy](#)
4. [Technology Stack & Justification](#)
5. [System Architecture](#)
6. [Database Implementation](#)

7. [Module Implementation Details](#)
 8. [User Interface Implementation](#)
 9. [Security Implementation](#)
 10. [Deployment Strategy](#)
 11. [Implementation Schedule](#)
 12. [Resource Requirements](#)
 13. [Risk Management](#)
 14. [Testing During Implementation](#)
 15. [Training & Handover Plan](#)
 16. [Conclusion](#)
 17. [References](#)
-

1. INTRODUCTION

1.1 Purpose

This Implementation Plan details the technical approach, tools, methodologies, and phased schedule adopted in building the **Matundu Primary School Management System (MATUNDU-SMS)**. It serves as the authoritative guide for translating the Design Specification into a working, deployable software product aligned with Kenya's Competency-Based Curriculum (CBC).

1.2 Scope

The plan covers the full-stack implementation of MATUNDU-SMS, including:

- Backend RESTful API development (Node.js + Express.js)
- Frontend single-page application (React 18 + Vite)
- PostgreSQL database schema with Prisma ORM
- Authentication & authorization (JWT + RBAC)
- 13 feature modules spanning academic, financial, and communication domains
- Cloud deployment on Render.com (backend) and Cloudflare Pages (frontend)
- Docker-based local development environment

1.3 Intended Audience

This document is intended for:

- **Project Supervisor** — Deborah Mbula
- **Academic Examiners** — KCA University, School of Computing & Informatics
- **Development Team** — For reference during future maintenance
- **School Administration** — Matundu Primary School stakeholders

1.4 Definitions & Acronyms

Acronym	Definition
CBC	Competency-Based Curriculum
RBAC	Role-Based Access Control
JWT	JSON Web Token
ORM	Object-Relational Mapping
REST	Representational State Transfer
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
SPA	Single Page Application
CI/CD	Continuous Integration / Continuous Deployment
STK	SIM Toolkit (M-Pesa)
KPI	Key Performance Indicator
CORS	Cross-Origin Resource Sharing

2. SYSTEM OVERVIEW

2.1 Project Summary

MATUNDU-SMS is a **production-grade, full-stack web application** designed to digitize and streamline operations at Matundu Primary School. The system manages the full lifecycle of school administration — from student admissions and staff management through to CBC-aligned assessments, report card generation, fee collection, and stakeholder communication.

2.2 Core Functional Modules

The system comprises **13 feature modules**:

#	Module	Description
1	Authentication	User login, registration, JWT tokens, password management
2	Student Management	Admissions, profiles, guardians, promotion workflow
3	Staff Management	Teacher profiles, qualifications, subject assignments
4	Academic Structure	Academic years, terms, grades (PP1–Grade 9), classes, subjects
5	Timetable	Weekly lesson scheduling with teacher-slot linking
6	Attendance	Daily attendance marking with QR code support
7	Assessments	CATs, exams, CBC competency scoring (EE/ME/AE/BE)
8	Reports	Report cards with rankings, PDF generation
9	Fee Management	Fee structures, invoicing, payments, M-Pesa integration
10	Communication	School announcements, direct messaging
11	AI Chat Assistant	Student-facing AI-powered study support
12	Integrations	Google Calendar & Microsoft Tasks connectivity
13	Dashboard	Analytics, KPIs, charts, and system overview

2.3 User Roles

The system supports **six distinct user roles**, each with tailored access:

Role	Access Level
SUPER_ADMIN	Full system access, configuration, and audit logs
ADMIN	School-wide management of students, staff, fees
TEACHER	Class management, assessments, attendance, reports
BURSAR	Fee structures, invoicing, payment recording
STUDENT	View own records, assessments, report cards, AI chat
PARENT	View child's academic performance and fee status

3. IMPLEMENTATION STRATEGY

3.1 Development Methodology

The project follows an **Agile-Iterative development approach** with incremental delivery across six defined phases. Each phase produces a working, testable increment of the system.

Key principles adopted:

- **Feature-based modular architecture** — Each module is self-contained with its own routes, controllers, services, and validators
- **API-first development** — Backend APIs are built and verified before frontend integration
- **Database-driven design** — Prisma ORM schema serves as the single source of truth for data models
- **Continuous deployment** — Automated deployment pipelines to Render.com and Cloudflare Pages

3.2 Development Phases Overview

Phase	Name	Focus Area	Status
1	Foundation	Auth, database, core entities	 Complete
2	Core Academic	Timetable, attendance, assessments	 Complete
3	Finance & Communication	Fees, payments, messaging	 Complete
4	Advanced Features	AI chat, QR attendance, integrations	 90%
5	Testing & Deployment	Automated tests, CI/CD, production	 70%
6	Mobile & Expansion	Flutter app, SMS, multi-school	 Planned

3.3 Coding Standards

The following coding standards are enforced throughout the codebase:

- **ES Module syntax** (`import/export`) — both backend and frontend use `"type": "module"`
- **Feature-based folder structure** — grouped by domain, not by file type
- **MVC pattern** — Routes → Controllers → Services → Prisma queries
- **Consistent naming** — camelCase for variables/functions, PascalCase for models/components
- **Environment-based configuration** — All secrets via `.env` files, never hardcoded
- **Error handling** — Centralized error handler middleware with structured error responses

4. TECHNOLOGY STACK & JUSTIFICATION

4.1 Frontend Technologies

Technology	Version	Purpose	Justification
React	18.3.1	UI component library	Industry-standard SPA framework; component reusability; large ecosystem
Vite	7.3.1	Build tool & dev server	Extremely fast HMR (Hot Module Replacement); ES module-native builds
Tailwind CSS	3.4.15	Utility-first styling	Rapid UI development; consistent design system; small bundle size
Zustand	5.0.1	Global state management	Lightweight alternative to Redux; minimal boilerplate
React Router DOM	6.28.0	Client-side routing	Declarative routing with nested layout support
React Hook Form	7.53.2	Form state management	Performant form handling with minimal re-renders
Recharts	2.13.3	Data visualization	React-native charting library for dashboard analytics
Lucide React	0.460.0	Icon library	Modern, consistent SVG icon set; tree-shakeable
jsPDF + AutoTable	4.1.0	Client-side PDF generation	Report card PDF export without server round-trip
Axios	1.7.7	HTTP client	Promise-based; interceptors for auth token injection

4.2 Backend Technologies

Technology	Version	Purpose	Justification
Node.js	20+	Server runtime	Non-blocking I/O; JavaScript full-stack consistency
Express.js	4.21.1	Web framework	Mature, minimal, highly extensible middleware ecosystem
Prisma	5.22.0	Database ORM	Type-safe queries; auto-generated client; migration system
PostgreSQL	15+	Relational database	ACID compliance; robust for complex relational data
Redis	5.10.0	Caching & rate limiting	In-memory store for session caching and API rate limiting
JWT (jsonwebtoken)	9.0.2	Authentication tokens	Stateless auth; widely supported; refresh token rotation
bcryptjs	2.4.3	Password hashing	Adaptive hashing with configurable salt rounds (12)
Helmet	8.0.0	HTTP security headers	Protection against common web vulnerabilities
Winston	3.15.0	Logging	Structured logging with multiple transport support
PDFKit	0.15.0	Server-side PDF generation	Programmatic PDF creation for report cards
Multer	1.4.5	File upload handling	Multipart form data processing for course resources
Swagger	6.2.8	API documentation	Auto-generated interactive API docs from JSDoc comments

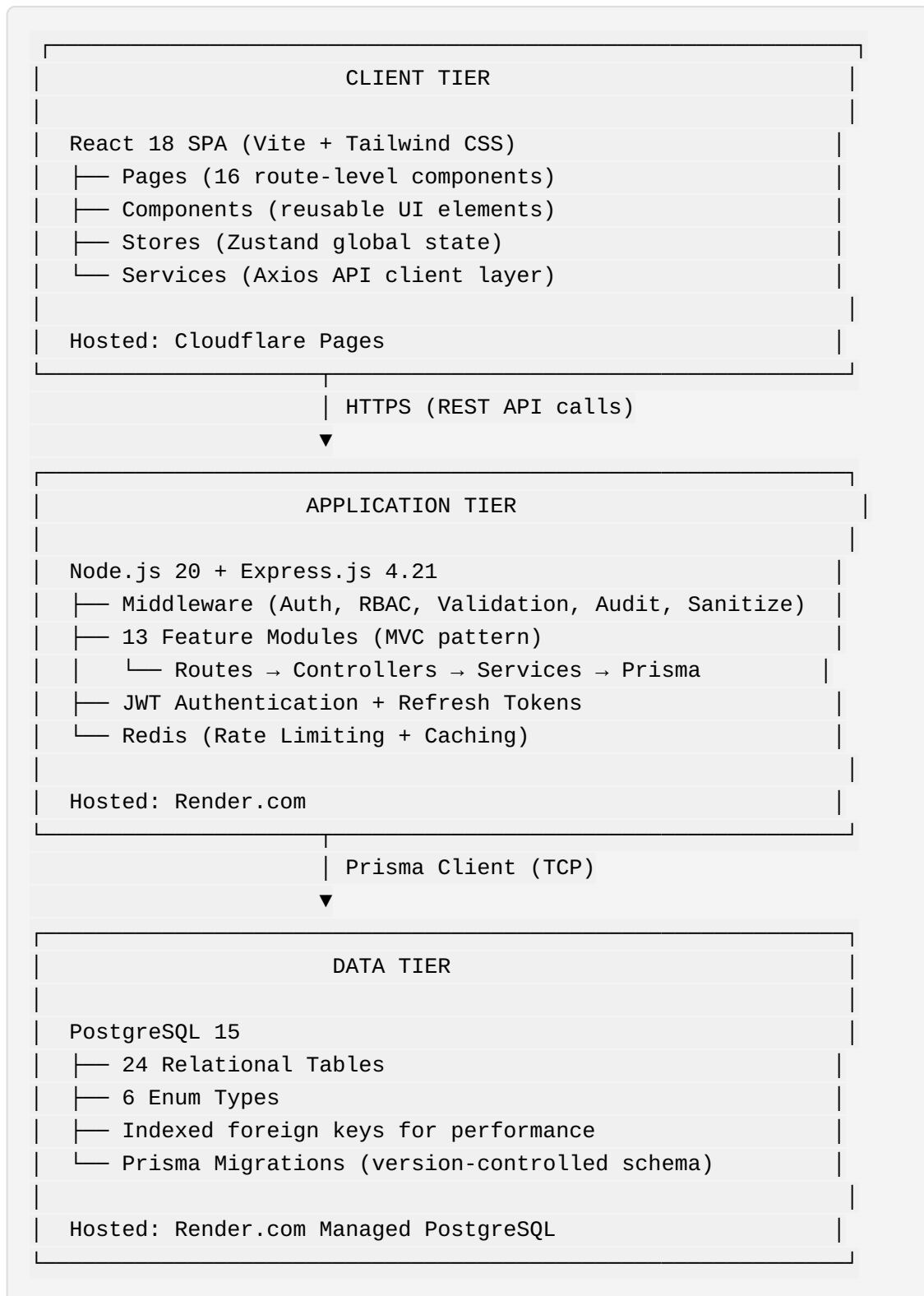
4.3 DevOps & Deployment

Technology	Purpose	Justification
Docker + Docker Compose	Local development environment	Consistent, reproducible dev setup
Render.com	Backend hosting	Free tier; auto-deploy from GitHub; managed PostgreSQL
Cloudflare Pages	Frontend hosting	Global CDN; fast builds; free SSL
GitHub Actions	CI/CD pipelines	Automated testing and deployment on push

5. SYSTEM ARCHITECTURE

5.1 High-Level Architecture

The system follows a **three-tier client-server architecture**:



5.2 Backend Module Architecture

Each feature module follows a consistent **MVC pattern**:

```

feature/
├─ feature.routes.js      # Express route definitions + middleware guards
├─ feature.controller.js  # Request/Response handling (HTTP layer)
├─ feature.service.js     # Business logic + Prisma database queries
└─ feature.validator.js   # Input validation rules (express-validator)

```

This pattern ensures **separation of concerns**, making modules independently testable and maintainable.

5.3 Frontend Component Architecture

```

frontend/src/
├─ App.jsx                # Root component with React Router v6 routes
├─ main.jsx               # Application entry point (ReactDOM.createRoot)
├─ index.css              # Global styles + Tailwind directives
├─ components/
│   ├─ Layout.jsx         # Application shell (sidebar + topbar)
│   ├─ common/            # Shared reusable components
│   └─ dashboard/         # Dashboard-specific widgets
├─ pages/                 # 16 route-level page components
├─ services/              # Axios-based API service layer
├─ stores/                # Zustand global state stores
└─ utils/                 # Helper functions and constants

```

5.4 API Design Pattern

All API endpoints follow RESTful conventions:

- **Base URL:** `/api/{resource}`
- **HTTP Methods:** GET (read), POST (create), PUT/PATCH (update), DELETE (remove)
- **Authentication:** Bearer token in `Authorization` header
- **Response format:** Consistent JSON envelope: `{ success: true, data: {...}, message: "..."}`
- **Error format:** `{ success: false, error: { code, message, details } }`
- **Pagination:** Query parameters `?page=1&limit=20`

6. DATABASE IMPLEMENTATION

6.1 Database Design

The PostgreSQL database schema contains **24 models** and **6 enum types**, implemented using Prisma ORM with version-controlled migrations.

6.2 Entity-Relationship Summary

Core Entities:

Entity	Key Fields	Relationships
User	id, email, phone, password, role, isActive	1:1 → Student, Staff, Guardian
Student	admissionNumber, dateOfBirth, admissionStatus	M:M → Guardian; 1:M → Class
Staff	employeeNumber, qualification, specialization	1:M → TeacherAssignment
Guardian	relationship, occupation, address	M:M → Student (via StudentGuardian)

Academic Entities:

Entity	Key Fields	Relationships
AcademicYear	name, startDate, endDate, isCurrent	1:M → Term, Class
Term	name (Term 1/2/3), startDate, endDate	M:1 → AcademicYear
Grade	name (PP1–Grade 9), level	1:M → Class, Subject
Class	name	M:1 → Grade, Stream, AcademicYear
Subject	name, code	M:M → Class (via ClassSubject)
TimetableSlot	day, startTime, endTime	M:1 → Class, Subject, Staff

Assessment Entities:

Entity	Key Fields	Relationships
Assessment	name, type (CAT/EXAM), maxScore, weight	1:M → AssessmentScore
AssessmentScore	score, grade	M:1 → Assessment, Student
Competency	name, description	1:M → CompetencyScore
CompetencyScore	rating (EE/ME/AE/BE)	M:1 → Competency, Student
ReportCard	rank, average, comments	M:1 → Student, Term, Class

Financial Entities:

Entity	Key Fields	Relationships
FeeStructure	amount, description	M:1 → Grade, Term
StudentInvoice	totalAmount, amountPaid, balance	M:1 → Student; 1:M → InvoiceItem
Payment	amount, method, transactionRef	M:1 → StudentInvoice

6.3 Migration Strategy

Prisma migrations are used to manage database schema evolution:

```
# Create a new migration
npx prisma migrate dev --name migration_name

# Apply migrations to production
npx prisma migrate deploy

# Seed initial data (admin user, grades, subjects)
npm run db:seed
```

6.4 Database Seeding

The seed script (`prisma/seed.js`) populates:

- Default admin user (admin@matundu.ac.ke)
 - CBC grade levels (PP1, PP2, Grade 1 through Grade 9)
 - Academic year and term structure
 - Core subjects per grade level
 - Sample class streams
-

7. MODULE IMPLEMENTATION DETAILS

7.1 Authentication Module

Files: `backend/src/features/auth/`

Component	Implementation Detail
Registration	Email + password; bcrypt hashing (12 salt rounds)
Login	Email/password validation; returns access + refresh tokens
Access Token	JWT signed with <code>JWT_SECRET</code> ; 15-minute expiry
Refresh Token	JWT signed with <code>JWT_REFRESH_SECRET</code> ; 7-day expiry; stored in DB
Token Refresh	Validates refresh token; issues new access token
Password Change	Requires current password verification before update
Rate Limiting	15 requests max per window on auth endpoints (Redis-backed)

Frontend: `Login.jsx` — Clean login form with email/password fields, error display, and JWT token storage in Zustand.

7.2 Student Management Module

Files: `backend/src/features/students/`

Feature	Implementation
Student Registration	Creates User (STUDENT role) + Student record atomically
Admission Workflow	PENDING → APPROVED / REJECTED status transitions
Guardian Linking	Many-to-many relationship via StudentGuardian table
Student Profiles	Full profile with photo, DOB, class assignment
Class Promotion	Bulk promotion from current grade to next grade
Search & Filter	Paginated list with search by name, admission number

Frontend: `Students.jsx` (list view) + `StudentDetail.jsx` (full profile with tabs).

7.3 Staff Management Module

Files: `backend/src/features/staff/`

Feature	Implementation
Staff CRUD	Create, read, update, delete staff records
Teacher Assignments	Link teachers to specific classes and subjects
Workload Tracking	View assigned classes, subjects, and timetable slots
Qualification Storage	Store academic qualifications and specializations

Frontend: `Staff.jsx` — Staff list with assignment management interface.

7.4 Academic Structure Module

Files: `backend/src/features/academic/` (12 files — largest module)

Feature	Implementation
Academic Years	CRUD with <code>isCurrent</code> flag for active year
Terms	Three terms per year (Term 1, 2, 3) with date ranges
Grades	CBC levels: PP1, PP2, Grade 1–9
Streams	Class streams (A, B, etc.) per grade
Classes	Composite: Grade + Stream + AcademicYear
Subjects	Subject codes linked to specific grades
Class-Subject Map	Many-to-many linking of subjects to classes
Timetable	Weekly slot scheduling: day, time, class, subject, teacher
Course Outlines	JSON-based weekly lesson plans per subject/class/term
Course Resources	File uploads (PDF, DOCX) and external links per subject

Frontend: `Classes.jsx` + `Timetable.jsx` — Visual timetable grid with management.

7.5 Attendance Module

Files: `backend/src/features/attendance/` (5 files)

Feature	Implementation
Daily Marking	Teacher marks each student: PRESENT, ABSENT, LATE, EXCUSED
Class Attendance	View attendance for a class on a specific date
Student History	Individual attendance history with statistics
QR Code Check-in	Generate QR codes for automated student check-in
Reports	Attendance percentage calculations per student/class

Frontend: `Attendance.jsx` — Class-based daily attendance marking interface.

7.6 Assessment & Grading Module

Files: `backend/src/features/assessments/`

Feature	Implementation
Assessment Creation	CAT, Exam, or Assignment with max score and weight
Score Recording	Bulk score entry per assessment per class
CBC Competencies	Competency definitions linked to subjects
Competency Scoring	Rate students: EE, ME, AE, BE per competency
Grade Calculation	Automatic grade assignment based on percentage score

CBC Grading Scale Implementation:

Rating	Label	Percentage Range
EE	Exceeding Expectations	75% and above
ME	Meeting Expectations	50% – 74%
AE	Approaching Expectations	25% – 49%
BE	Below Expectations	Below 25%

Traditional Grading Scale:

Grade	Label	Percentage Range
A	Excellent	80% and above
B	Good	70% – 79%
C	Average	60% – 69%
D	Below Average	50% – 59%
E	Poor	Below 50%

Frontend: `Assessments.jsx` — Assessment creation, bulk score entry with class/subject selectors.

7.7 Report Card Module

Files: `backend/src/features/reports/`

Feature	Implementation
Report Generation	Aggregates scores across assessments for a term
Class Rankings	Rank students by weighted average within a class
Mean Score Calc	Class mean, student mean, subject-wise breakdown
PDF Export (Backend)	PDFKit-generated report cards with school branding
PDF Export (Frontend)	jsPDF + AutoTable client-side generation
Comments	Teacher and principal comment fields per report

Frontend: `Reports.jsx` (50.4KB — most complex page) — Report card viewer with PDF download.

7.8 Fee Management Module

Files: `backend/src/features/fees/` (4 files)

Feature	Implementation
Fee Structures	Define amounts per grade per term
Invoice Generation	Auto-generate invoices per student per term
Invoice Items	Line-item breakdown (tuition, lunch, transport, etc.)
Payment Recording	Record payments via Cash, M-Pesa, or Bank Transfer
Balance Tracking	Auto-calculate outstanding balance per student
M-Pesa STK Push	Safaricom Daraja API integration for mobile payments
Payment Callback	Webhook endpoint for M-Pesa payment confirmation

Frontend: `Fees.jsx` — Fee structure management, invoice viewer, payment recording interface.

7.9 Communication Module

Files: `backend/src/features/communication/`

Feature	Implementation
Announcements	School-wide announcements with role-based targeting
Direct Messaging	User-to-user messaging within the platform
Read Receipts	Track message read status

Frontend: `Announcements.jsx` — Announcement creation and viewing interface.

7.10 Dashboard & Analytics

Files: `backend/src/features/dashboard/`

Feature	Implementation
KPI Cards	Total students, staff, classes, attendance rate
Enrollment Chart	Student enrollment trends over academic years
Attendance Chart	Daily/weekly attendance visualization
Fee Collection	Revenue tracking and outstanding balances
Grade Distribution	Assessment performance distribution per class
System Health	API uptime and response time monitoring

Frontend: `Dashboard.jsx` (40.2KB) — Rich analytics hub with Recharts-powered visualizations.

7.11 AI Chat Assistant

Files: `backend/src/features/ai/`

Feature	Implementation
Chat Interface	Student-facing conversational AI
Chat History	Persistent conversation storage per student
AI Model	Gemini Pro integration via API key
Context Awareness	Understands student's enrolled subjects and curriculum

7.12 Third-Party Integrations

Files: `backend/src/features/integrations/`

Integration	Implementation
Google Calendar	OAuth 2.0 flow; create/sync school events
Microsoft Tasks	MS Graph API; sync assignments and deadlines
Google OAuth	OAuth callback handling and token storage
Microsoft OAuth	OAuth callback handling and token storage

8. USER INTERFACE IMPLEMENTATION

8.1 Page Inventory

The frontend consists of **16 fully implemented pages**:

#	Page	File	Size	Access Roles
1	Login	Login.jsx	3.9 KB	Public
2	Dashboard	Dashboard.jsx	40.2 KB	All authenticated
3	Students	Students.jsx	11.9 KB	Admin, Teacher
4	Student Detail	StudentDetail.jsx	20.3 KB	Admin, Teacher
5	Staff	Staff.jsx	11.8 KB	Admin
6	Classes	Classes.jsx	14.7 KB	Admin, Teacher
7	Assessments	Assessments.jsx	21.6 KB	Teacher
8	Attendance	Attendance.jsx	10.8 KB	Teacher
9	Fees	Fees.jsx	21.3 KB	Bursar, Admin
10	Reports	Reports.jsx	50.4 KB	Admin, Teacher
11	Timetable	Timetable.jsx	22.1 KB	Admin, Teacher
12	Announcements	Announcements.jsx	12.9 KB	All authenticated
13	Admissions	Admissions.jsx	10.2 KB	Admin
14	Settings	Settings.jsx	22.7 KB	Admin
15	Profile	Profile.jsx	3.7 KB	All authenticated
16	System Status	SystemStatus.jsx	6.7 KB	Admin

8.2 UI/UX Design Principles

- **Responsive Design:** Tailwind CSS breakpoints ensure usability on desktop, tablet, and mobile
- **Role-Based Navigation:** Sidebar menu items dynamically filtered by user role
- **Consistent Layout:** Shared `Layout.jsx` shell with sidebar navigation and top bar
- **Interactive Feedback:** React Hot Toast for success/error notifications

- **Form Validation:** React Hook Form with real-time client-side validation
- **Data Visualization:** Recharts library for interactive charts and graphs
- **Loading States:** Skeleton loaders and spinners during API calls

8.3 State Management

Zustand stores provide lightweight global state:

- **Auth Store** — User session, tokens, role
- **UI Store** — Sidebar state, theme preferences
- **Data Stores** — Cached API responses per module

9. SECURITY IMPLEMENTATION

9.1 Authentication Security

Measure	Implementation
Password Hashing	bcrypt with 12 salt rounds
Access Tokens	JWT with 15-minute expiry
Refresh Tokens	JWT with 7-day expiry; stored in database
Token Rotation	New refresh token issued on each refresh
Token Revocation	Database-backed; can invalidate specific tokens

9.2 Authorization Security

Measure	Implementation
RBAC Middleware	Role verification on every protected endpoint
Resource-Level Access	Fine-grained access control per resource
Route Guards (Frontend)	Protected routes redirect unauthorized users

9.3 Application Security

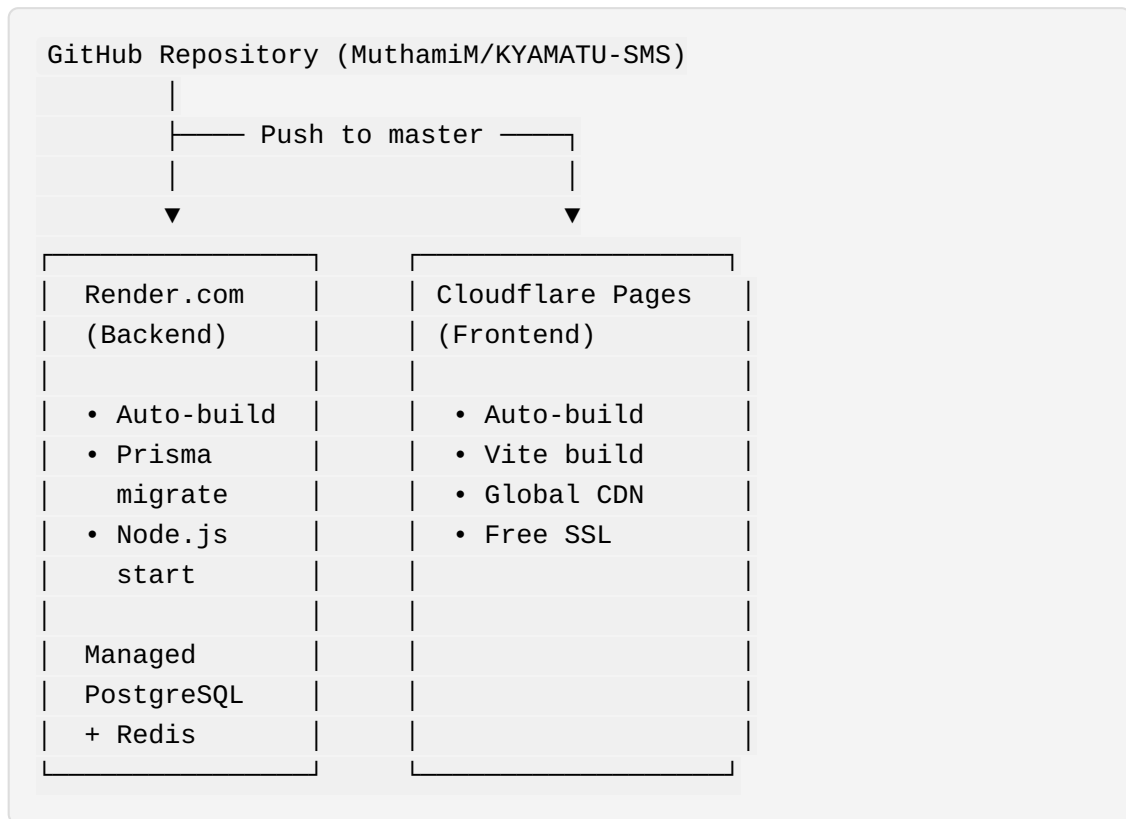
Measure	Implementation
Helmet.js	Sets secure HTTP headers (CSP, HSTS, X-Frame, etc.)
CORS Configuration	Whitelist specific frontend origin
Rate Limiting	Redis-backed rate limiter on auth endpoints (15 req/window)
Input Sanitization	Custom sanitize middleware on all mutation routes
Input Validation	express-validator on all POST/PUT endpoints
SQL Injection Prevention	Prisma ORM parameterized queries (no raw SQL)
Audit Logging	All mutations recorded with old/new values

9.4 Environment Security

Measure	Implementation
Secret Management	All secrets in <code>.env</code> files; never committed to Git
<code>.gitignore</code>	Excludes <code>.env</code> , <code>node_modules</code> , build artifacts
HTTPS	Enforced on both Render.com and Cloudflare Pages

10. DEPLOYMENT STRATEGY

10.1 Deployment Architecture



10.2 Backend Deployment (Render.com)

Configuration	Value
Service Type	Web Service (Node.js)
Region	Oregon (us-west)
Build Command	<code>npm install && npx prisma generate</code>
Start Command	<code>npx prisma migrate deploy && npm start</code>
Database	Managed PostgreSQL (<code>matundu-db</code>)
Cache	Managed Redis (<code>matundu-redis</code>)
Auto-Deploy	On push to <code>master</code> branch

10.3 Frontend Deployment (Cloudflare Pages)

Configuration	Value
Framework	Vite
Build Command	<code>npm run build</code>
Output Directory	<code>dist</code>
Configuration File	<code>frontend/wrangler.toml</code>
Auto-Deploy	On push to <code>master</code> branch

10.4 Local Development (Docker)

```
# docker-compose.yml services:  
# PostgreSQL → port 5432  
# Backend API → port 3000  
# Frontend → port 5173
```

```
# Start all services  
docker-compose up -d  
  
# Seed the database  
cd backend && npm run db:seed
```

11. IMPLEMENTATION SCHEDULE

11.1 Phase 1: Foundation (Weeks 1–3) COMPLETE

Week	Task	Deliverable
1	Project setup, tooling, Docker config	Development environment ready
1	PostgreSQL + Prisma schema design	24-model database schema
2	Express.js app scaffolding	Middleware stack configured
2	Authentication module (JWT + bcrypt)	Login/register/refresh APIs
2	RBAC middleware implementation	6 roles enforced
3	Student management module	Full CRUD + admissions workflow
3	Staff management module	CRUD + teacher assignments
3	Academic structure (grades, classes)	CBC grade levels seeded

11.2 Phase 2: Core Academic (Weeks 4–6) COMPLETE

Week	Task	Deliverable
4	Timetable module implementation	Weekly scheduling API + grid UI
4	Subject management and class-subject linking	Subject assignment system
5	Daily attendance module	Teacher attendance marking
5	Assessment creation and score recording	CAT/Exam management
6	CBC competency scoring	EE/ME/AE/BE rating system
6	Report card generation	Rankings, averages, PDF export
6	Course outlines and resources	Lesson planning tools

11.3 Phase 3: Finance & Communication (Weeks 7–9) COMPLETE

Week	Task	Deliverable
7	Fee structure management	Grade/term fee definitions
7	Invoice generation system	Auto-generated student invoices
8	Payment recording (Cash, Bank, M-Pesa)	Payment tracking system
8	M-Pesa STK Push integration	Safaricom Daraja API connected
9	Announcements system	Role-targeted announcements
9	Direct messaging	User-to-user messaging

11.4 Phase 4: Advanced Features (Weeks 10–12) 90% COMPLETE

Week	Task	Deliverable
10	AI chat assistant backend	Gemini Pro integration
10	QR code attendance service	QR generation and scanning
11	Google Calendar integration	OAuth + event sync
11	Microsoft Tasks integration	MS Graph API connectivity
12	Audit logging middleware	Full mutation trail
12	Dashboard analytics	KPI cards + Recharts charts

11.5 Phase 5: Testing & Deployment (Weeks 13–15) 🟡 70% COMPLETE

Week	Task	Deliverable
13	Docker Compose finalization	Local dev environment ready
13	Render.com backend deployment	Live API endpoint
14	Cloudflare Pages frontend deployment	Live frontend URL
14	GitHub Actions CI/CD setup	Automated deploy pipeline
15	Integration testing	End-to-end workflow verification
15	Production smoke testing	System stability confirmed

11.6 Phase 6: Mobile & Expansion (Planned — Future)

Task	Target
Flutter parent mobile app	Future release
SMS notification service	Africa's Talking integration
Multi-school tenancy support	Multi-tenant architecture
Advanced analytics dashboard	Enhanced data insights

12. RESOURCE REQUIREMENTS

12.1 Hardware Requirements

Development Environment:

Resource	Minimum Specification
Processor	Intel Core i5 / Apple M1 or equivalent
RAM	8 GB (16 GB recommended for Docker)
Storage	20 GB free disk space
Network	Stable internet connection for API testing

Production Environment (Cloud-Hosted):

Component	Provider	Specification
Backend Server	Render.com	Free tier web service (512 MB)
PostgreSQL	Render.com	Free tier managed database
Redis	Render.com	Free tier managed cache
Frontend CDN	Cloudflare Pages	Free tier with global CDN
Domain	Cloudflare	Custom domain (optional)

12.2 Software Requirements

Software	Version	Purpose
Node.js	20+	JavaScript runtime
npm	10+	Package manager
PostgreSQL	15+	Database server (or Docker)
Docker	24+	Containerized local development
Git	2.40+	Version control
VS Code	Latest	Primary IDE
Google Chrome	Latest	Testing and debugging

12.3 Human Resources

Role	Responsibility
Developer	Full-stack development and testing (Musa Muthami Mwange)
Project Supervisor	Guidance and review (Deborah Mbula)
School Stakeholders	Requirements validation and UAT feedback

13. RISK MANAGEMENT

13.1 Risk Assessment Matrix

#	Risk	Probability	Impact	Mitigation Strategy
1	M-Pesa API sandbox unavailability	Medium	High	Implement mock payment service for testing; document manual fallback
2	Database migration conflicts	Low	High	Use Prisma migration history; maintain backup SQL scripts
3	Third-party API changes (Google, MS)	Medium	Medium	Abstract integrations behind service layer; graceful degradation
4	Free-tier hosting limitations	High	Medium	Monitor usage quotas; plan upgrade path; implement caching
5	Data loss during deployment	Low	High	Automated database backups; <code>backups/</code> folder maintained
6	Security vulnerabilities	Medium	High	Regular <code>npm audit</code> ; Helmet.js; input sanitization; RBAC
7	Browser compatibility issues	Low	Medium	Test on Chrome, Firefox, Safari; Tailwind CSS ensures consistency
8	Scope creep (additional features)	High	Medium	Strict phase-based delivery; defer non-essential features to Phase 6
9	Performance degradation under load	Medium	Medium	Redis caching; database indexing; paginated API responses
10	AI API key exhaustion / cost overrun	Medium	Low	Rate limiting on AI chat endpoint; usage monitoring

13.2 Contingency Plans

- **Database Failure:** Restore from latest backup in `backups/` directory; Render.com provides automated backups
 - **Hosting Downtime:** Docker Compose enables instant local fallback environment
 - **API Dependency Failure:** All third-party integrations are optional; core system functions independently
-

14. TESTING DURING IMPLEMENTATION

14.1 Testing Approach

Testing is conducted in parallel with implementation at each phase:

Test Type	Tool / Method	Scope
Unit Testing	Vitest	Service layer functions
API Testing	Manual + Swagger UI	All REST endpoints
Integration Testing	End-to-end workflows	Cross-module data flow
UI Testing	Manual browser testing	All 16 pages across roles
Security Testing	<code>npm audit</code> + manual review	Dependency vulnerabilities
Performance Testing	Browser DevTools + API timing	Response time benchmarking
User Acceptance	Stakeholder demos	Feature validation with school staff

14.2 Test Credentials

Role	Email	Password
Admin	admin@matundu.ac.ke	Admin@123
Teacher	teacher@matundu.ac.ke	Admin@123
Bursar	bursar@matundu.ac.ke	Admin@123
Student	student1@matundu.ac.ke	Admin@123

Note: These are development/testing credentials only. All passwords must be changed before production deployment.

14.3 Key Test Scenarios

#	Scenario	Expected Outcome
1	Admin login and dashboard access	Dashboard loads with KPI cards/charts
2	Create new student with guardian	Student record created; admission PENDING
3	Approve student admission	Status changes to APPROVED
4	Teacher marks daily attendance	Attendance records saved successfully
5	Create CAT and enter scores	Scores saved; grades auto-calculated
6	Generate term report card	Report card with rankings generated
7	Export report card as PDF	PDF downloaded successfully
8	Create fee invoice for student	Invoice generated with correct balance
9	Record payment (Cash)	Balance updated; payment recorded
10	Create school announcement	Announcement visible to targeted roles

15. TRAINING & HANDOVER PLAN

15.1 User Training

User Group	Training Content	Duration
Administrators	System setup, user management, academic configuration, reports	2 days
Teachers	Attendance marking, assessment entry, report cards	1 day
Bursars	Fee structure setup, invoicing, payment recording	1 day
Students	Login, view grades, report cards, AI chat	30 minutes

15.2 Documentation Deliverables

Document	Status	Location
Proposal Document	 Complete	docs/1_Proposal_Document.pdf
Requirements Specification	 Complete	docs/ 2_Requirements_Specification_SRS.pdf
Design Specification	 Complete	docs/3_Design_Specification.pdf
Test Plan	 Complete	docs/4_Test_Plan.pdf
Implementation Plan	 Complete	docs/5_Implementation_Plan.pdf
User Manual	 Complete	docs/6_User_Manual.pdf
Final Project Report	 Complete	docs/7_Final_Project_Report.pdf
Final Presentation	 Complete	docs/8_Final_Presentation.pptx

15.3 Handover Checklist

- [x] Source code committed to GitHub repository
 - [x] Database migrations documented and versioned
 - [x] Environment variables template (`.env.example`) provided
 - [x] Deployment configurations (Render + Cloudflare) documented
 - [x] Docker Compose for local development verified
 - [x] Test credentials and seed data documented
 - [x] User Manual delivered
 - [] Training sessions conducted
-

16. CONCLUSION

The Matundu Primary School Management System has been implemented as a comprehensive, production-grade web application that successfully digitizes core school operations. The system covers **13 functional modules** spanning student management, CBC-aligned academic tracking, financial management, and stakeholder communication.

Key implementation achievements:

- **Full-stack architecture** — React 18 frontend + Node.js/Express backend + PostgreSQL database
- **CBC alignment** — Competency-based grading (EE/ME/AE/BE) fully integrated
- **Role-based security** — Six user roles with granular access control
- **Production deployment** — Live on Render.com (backend) and Cloudflare Pages (frontend)
- **Extensible design** — Modular feature-based architecture enables future enhancements

The system is positioned for continued development, with planned enhancements including a Flutter parent mobile application, SMS notifications via Africa's Talking, and multi-school tenancy support.

17. REFERENCES

1. Kenya Institute of Curriculum Development (KICD). (2017). *Competency-Based Curriculum Design*. Ministry of Education, Kenya.
2. Prisma Documentation. (2024). *Prisma ORM — Next-Generation Node.js and TypeScript ORM*. Retrieved from <https://www.prisma.io/docs>
3. React Documentation. (2024). *React — A JavaScript Library for Building User Interfaces*. Retrieved from <https://react.dev>
4. Express.js Documentation. (2024). *Express — Node.js Web Application Framework*. Retrieved from <https://expressjs.com>
5. Vite Documentation. (2024). *Vite — Next Generation Frontend Tooling*. Retrieved from <https://vitejs.dev>
6. PostgreSQL Documentation. (2024). *PostgreSQL: The World's Most Advanced Open Source Relational Database*. Retrieved from <https://www.postgresql.org/docs>
7. Tailwind CSS Documentation. (2024). *Tailwind CSS — Rapidly Build Modern Websites*. Retrieved from <https://tailwindcss.com/docs>
8. Safaricom Developer Portal. (2024). *M-Pesa Daraja API Documentation*. Retrieved from <https://developer.safaricom.co.ke>
9. JSON Web Tokens. (2024). *Introduction to JSON Web Tokens*. Retrieved from <https://jwt.io/introduction>
10. Docker Documentation. (2024). *Docker Compose Overview*. Retrieved from <https://docs.docker.com/compose>

Prepared by: Musa Muthami Mwange (23/05037) **Supervised by:** Deborah Mbula
Institution: KCA University **Date:** October 2026

© 2026 Matundu Primary School Management System. All rights reserved.